AI ASSISTED CODING

Name: P.Swaran Raj **Roll.No:** 2303A52247

Batch: 37 **Assignment -** 20.2

Task 1 – Password Security Enhancement

Prompt:

"#Generate a Python program for user registration storing username and password."

Insecure Code:

```python
users_insecure = {}

def register_user_insecure(username, password):
    if username in users_insecure:
        return "Username already exists."
    else:
        users_insecure[username] = password # Storing password in plain text - INSECURE!
        return f"User '{username}' registered successfully!"

def login_user_insecure(username, password):
    if username not in users_insecure:
        return "Login failed: Username not found."
    else:
        if users_insecure[username] == password:
            return f"Login successful for user '{username}'!"
        else:
            return "Login failed: Incorrect password."

# --- Example Usage (Insecure) ---
print("--- Insecure Registration Examples ---")
print(register_user_insecure("john_doe", "mysecretpassword"))
print(register_user_insecure("jane_smith", "anothersecret"))
print(register_user_insecure("john_doe", "newpassword")) # Try registering existing user

print("\n--- Stored Users (Plain-text passwords) ---")
print(users_insecure)

print("\n--- Insecure Login Examples ---")
print(login_user_insecure("john_doe", "mysecretpassword")) # Correct password
print(login_user_insecure("jane_smith", "anothersecret"))   # Correct password
print(login_user_insecure("john_doe", "wrongpass"))      # Incorrect password
print(login_user_insecure("peter_pan", "anypass"))     # Non-existent user
```

Insecure Output:

```
--- Insecure Registration Examples ---
User 'john_doe' registered successfully!
User 'jane_smith' registered successfully!
Username already exists.

--- Stored Users (Plain-text passwords) ---
{'john_doe': 'mysecretpassword', 'jane_smith': 'anothersecret'}

--- Insecure Login Examples ---
Login successful for user 'john_doe'!
Login successful for user 'jane_smith'!
Login failed: Incorrect password.
Login failed: Username not found.
```

Improvised Prompt:

"#Refactor login system with plain-text password storage, identify its security flaws, and improve it by implementing secure hashing and authentication."

Secured Code:

```python
import hashlib

users_secure = {}

def hash_password(password):
    # Hash the password using SHA-256.
    # In a real application, also use salting and more robust algorithms like bcrypt.
    return hashlib.sha256(password.encode()).hexdigest()

def register_user_secure(username, password):
    if username in users_secure:
        return "Username already exists. Please choose a different username."
    else:
        hashed_password = hash_password(password)
        users_secure[username] = hashed_password
        return f"User '{username}' registered successfully!"

def login_user_secure(username, password):
    if username not in users_secure:
        return "Login failed: Username not found."
    else:
        hashed_input_password = hash_password(password)
        if users_secure[username] == hashed_input_password:
            return f"Login successful for user '{username}'!"
        else:
            return "Login failed: Incorrect password."

# --- Example Usage (Secure) ---
print("--- Secure Registration Examples ---")
print(register_user_secure("alice", "securepass123"))
print(register_user_secure("bob", "supersecurepass"))
print(register_user_secure("alice", "newsecurepass")) # Try registering existing user

print("\n--- Stored Users (Hashed passwords) ---")
print(users_secure)
```

```python
print("\n--- Secure Login Examples ---")
print(login_user_secure("alice", "securepass123")) # Correct password
print(login_user_secure("bob", "supersecurepass"))   # Correct password
print(login_user_secure("alice", "wrongpass"))    # Incorrect password
print(login_user_secure("charlie", "anypass"))  # Non-existent user
```

Insecure Output:

```
--- Secure Registration Examples ---
User 'alice' registered successfully!
User 'bob' registered successfully!
Username already exists. Please choose a different username.

--- Stored Users (Hashed passwords) ---
{'alice': '6e0daa0a792dcaef2473898426 7b05c5153663f16fa31a96325ab2fc1ca713b8', 'bob': '5e1cf737b6f4764ff22abc80925d1412443c4f2ecbfc5904143151d7bac6b6ec'}

--- Secure Login Examples ---
Login successful for user 'alice'!
Login successful for user 'bob'!
Login failed: Incorrect password.
Login failed: Username not found.
```

Explanation:

Passwords stored in plain text are insecure and can be easily exposed. The improved version uses hashing (SHA-256) to store passwords securely. During login, hashed values are compared instead of actual passwords, protecting user data.

Task 2 – Secure API Key Management

Prompt:

"#Generate a Python script that connects to an API using a hardcoded API key."

Insecure Code:

```python
import requests

# --- WARNING: HARDCODED API KEY ---
# Replace with your actual API key and endpoint
API_KEY = "your_hardcoded_api_key_here"
API_ENDPOINT = "https://jsonplaceholder.typicode.com/posts/1" # Example public API endpoint

# For APIs that require the key in the header (common)
headers = {
    "Authorization": f"Bearer {API_KEY}", # Or "X-API-Key": API_KEY, depending on the API
    "Content-Type": "application/json"
}

# For APIs that require the key as a query parameter
params = {
    "api_key": API_KEY
}

try:
    # Example: Making a GET request with key in headers
    print("\n--- Attempting GET request with API key in headers ---")
    response_headers = requests.get(API_ENDPOINT, headers=headers)
    response_headers.raise_for_status() # Raise an HTTPError for bad responses (4xx or 5xx)
    print("Response from API (headers):")
    print(response_headers.json())

    # Example: Making a GET request with key as a query parameter (if API supports it)
    print("\n--- Attempting GET request with API key as query parameter ---")
    response_params = requests.get(API_ENDPOINT, params=params)
    response_params.raise_for_status() # Raise an HTTPError for bad responses (4xx or 5xx)
    print("Response from API (query params):")
    print(response_params.json())

except requests.exceptions.RequestException as e:
    print(f"An error occurred: {e}")
    if hasattr(e, 'response') and e.response is not None:
```

Insecure Output:

```python
    if hasattr(e, 'response') and e.response is not None:
        print(f"Response content: {e.response.text}")
```

```
--- Attempting GET request with API key in headers ---
Response from API (headers):
{'userId': 1, 'id': 1, 'title': 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit', 'body': 'quia et suscipit\nsuscipit recusandae consequuntur exp

--- Attempting GET request with API key as query parameter ---
Response from API (query params):
{'userId': 1, 'id': 1, 'title': 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit', 'body': 'quia et suscipit\nsuscipit recusandae consequuntur exp
```

Improvised Prompt:

"#Refactor the Python script with a hardcoded API key, identify its security risks, and refactor it to use environment variables for secure API key management."

Improvised Code:

```python
import requests
import os

# --- Securely retrieving API Key from Environment Variable ---
# In Colab, you can set environment variables using %env magic command (for current session)
# or use the 'Secrets' tab (🔑 icon on the left panel) for persistent storage.
# For this example, let's assume an environment variable named 'MY_API_KEY' is set.
API_KEY = os.environ.get("MY_API_KEY", "YOUR_DEFAULT_API_KEY_OR_HANDLE_ERROR")
API_ENDPOINT = "https://jsonplaceholder.typicode.com/posts/1" # Example public API endpoint

if API_KEY == "YOUR_DEFAULT_API_KEY_OR_HANDLE_ERROR":
    print("Warning: MY_API_KEY environment variable not set. Using a placeholder. Please set it for actual API calls.")

# For APIs that require the key in the header (common)
headers = {
    "Authorization": f"Bearer {API_KEY}", # Or "X-API-Key": API_KEY, depending on the API
    "Content-Type": "application/json"
}

# For APIs that require the key as a query parameter
params = {
    "api_key": API_KEY
}

try:
    # Example: Making a GET request with key in headers
    print("\n--- Attempting GET request with API key in headers ---")
    response_headers = requests.get(API_ENDPOINT, headers=headers)
    response_headers.raise_for_status() # Raise an HTTPError for bad responses (4xx or 5xx)
    print("Response from API (headers):")
    print(response_headers.json())

    # Example: Making a GET request with key as a query parameter (if API supports it)
    print("\n--- Attempting GET request with API key as query parameter ---")
    response_params = requests.get(API_ENDPOINT, params=params)
    response_params.raise_for_status() # Raise an HTTPError for bad responses (4xx or 5xx)
```

```python
    print("Response from API (query params):")
    print(response_params.json())

except requests.exceptions.RequestException as e:
    print(f"An error occurred: {e}")
    if hasattr(e, 'response') and e.response is not None:
        print(f"Response content: {e.response.text}")
```

Secured Output:

```
Warning: MY_API_KEY environment variable not set. Using a placeholder. Please set it for actual API calls.

--- Attempting GET request with API key in headers ---
Response from API (headers):
{'userId': 1, 'id': 1, 'title': 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit', 'body': 'quia et suscipit\nsuscipit recusandae consequuntur expe

--- Attempting GET request with API key as query parameter ---
Response from API (query params):
{'userId': 1, 'id': 1, 'title': 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit', 'body': 'quia et suscipit\nsuscipit recusandae consequuntur expe
```

Explanation:

Hardcoded API keys are insecure because they can be exposed in the source code. Using environment variables hides sensitive data and improves security by preventing unauthorized access.

Task 3 – Weak Encryption Detection

Prompt:

"#Generate a Python program for encryption and decryption using a basic method."

Normal Code:

```python
def caesar_cipher(text, shift, mode='encrypt'):
    result = ""
    if mode == 'decrypt':
        shift = -shift

    for char in text:
        if char.isalpha():  # Check if the character is an alphabet letter
            start = ord('a') if char.islower() else ord('A')
            shifted_char = chr((ord(char) - start + shift) % 26 + start)
            result += shifted_char
        else:
            result += char  # Non-alphabetic characters are not changed
    return result

# --- Example Usage ---
original_text = "Hello, World! This is a secret message."
shift_key = 3

print(f"Original Text: {original_text}")
print(f"Shift Key: {shift_key}")

# Encryption
encrypted_text = caesar_cipher(original_text, shift_key, mode='encrypt')
print(f"Encrypted Text: {encrypted_text}")

# Decryption
decrypted_text = caesar_cipher(encrypted_text, shift_key, mode='decrypt')
print(f"Decrypted Text: {decrypted_text}")

# Another example
text2 = "Python Programming"
shift2 = 5
encrypted2 = caesar_cipher(text2, shift2)
print(f"\nOriginal: {text2}, Shift: {shift2}")
print(f"Encrypted: {encrypted2}")
print(f"Decrypted: {caesar_cipher(encrypted2, shift2, mode='decrypt')}")
```

Output:

```
Original Text: Hello, World! This is a secret message.
Shift Key: 3
Encrypted Text: Khoor, Zruog! Wklv lv d vhfuhw phvvdjh.
Decrypted Text: Hello, World! This is a secret message.

Original: Python Programming, Shift: 5
Encrypted: Udymts Uwtlwfrrnsl
Decrypted: Python Programming
```

Improvised Prompt:

"#Generate a Python encryption program using a weak method, identify its security flaws, and refactor it using a stronger encryption technique like Fernet."

Refactored Code:

```python
from cryptography.fernet import Fernet

# --- Key Generation ---
# In a real application, you would generate a key once and securely store it.
# NEVER hardcode or regenerate the key for every encryption/decryption operation.
# For demonstration, we generate a new key each time.
key = Fernet.generate_key()
fernet = Fernet(key)

print(f"Generated Fernet Key: {key.decode()}")

# --- Encryption ---
message_fernet = b"This is a super secret message using Fernet."
encrypted_message_fernet = fernet.encrypt(message_fernet)

print(f"\nOriginal Message (Fernet): {message_fernet.decode()}")
print(f"Fernet Encrypted: {encrypted_message_fernet.decode()}")

# --- Decryption ---
decrypted_message_fernet = fernet.decrypt(encrypted_message_fernet)

print(f"Fernet Decrypted: {decrypted_message_fernet.decode()}")

# --- Tampering attempt (will fail decryption due to HMAC) ---
print("\n--- Demonstrating Tampering Detection ---")
# Modify one byte of the encrypted message
tampered_message = bytearray(encrypted_message_fernet)
if len(tampered_message) > 10: # Ensure there's enough bytes to tamper with
    tampered_message[10] = (tampered_message[10] + 1) % 256 # Change a byte
tampered_message = bytes(tampered_message)

print(f"Tampered Message (first 20 bytes): {tampered_message[:20]}")

try:
    fernet.decrypt(tampered_message)
except Exception as e:
    print(f"Decryption of tampered message failed as expected: {e}")
```

Output:

```
Generated Fernet Key: pvEj3nyTlGoTVqLdce4vn5Gg20nVcDSz1MiZ1XCg00s=

Original Message (Fernet): This is a super secret message using Fernet.
Fernet Encrypted: gAAAAABpy1W4BsgM2UfJbKqaUySG9TPlF2OnqbsR8Bn_d3DVbIqUgsN7mOKsM8RSvRUHVT2EPiMzEI83bDw4xvD1AmBIR8LygrjTxiS-lb6YSzl-uMrzFz8PeScQfHyqjqtSiyzg9Z1q
Fernet Decrypted: This is a super secret message using Fernet.

--- Demonstrating Tampering Detection ---
Tampered Message (first 20 bytes): b'gAAAAABpy1X4BsgM2UfJ'
Decryption of tampered message failed as expected:
```

Explanation:

Basic methods like Base64 are not secure as they can be easily reversed. The improved version uses strong encryption (Fernet), which protects data and requires a key for decryption, making it more secure.

Task 4 – Real-Time Application: Security Review of AI-Generated Web Service

Prompt:

"#Generate a simple Python web service (Flask API) for user login."

Insecure Code:

```
!pip install Flask
```

```
Requirement already satisfied: Flask in /usr/local/lib/python3.12/dist-packages (3.1.3)
Requirement already satisfied: blinker>=1.9.0 in /usr/local/lib/python3.12/dist-packages (from Flask) (1.9.0)
Requirement already satisfied: click>=8.1.3 in /usr/local/lib/python3.12/dist-packages (from Flask) (8.3.1)
Requirement already satisfied: itsdangerous>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from Flask) (2.2.0)
Requirement already satisfied: jinja2>=3.1.2 in /usr/local/lib/python3.12/dist-packages (from Flask) (3.1.6)
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from Flask) (3.0.3)
Requirement already satisfied: werkzeug>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from Flask) (3.1.6)
```

```
import subprocess
import threading
from flask import Flask, request, jsonify

# Helper to kill any process using the port, ensuring Flask can start.
def kill_process_on_port(port):
    try:
        command = f"lsof -t -i:{port}"
        process = subprocess.run(command, shell=True, capture_output=True, text=True)
        pids = process.stdout.strip().split('\n')

        if pids and pids[0]:
            for pid in pids:
                if pid:
                    print(f"Killing process {pid} on port {port}...")
                    subprocess.run(f"kill -9 {pid}", shell=True)
            print(f"Processes on port {port} killed successfully.")
        else:
            print(f"No process found running on port {port}.")
    except Exception as e:
        print(f"Error killing process on port {port}: {e}")

# Kill any existing Flask app on port 5000 before starting a new one
kill_process_on_port(5000)

app = Flask(__name__)
```

```python
users_insecure_app = {
    "testuser": "password123",
    "admin": "adminpass"
}

@app.route('/login', methods=['POST'])
def login_insecure():
    data = request.get_json()
    username = data.get('username')
    password = data.get('password')

    # No input validation and plain text password comparison - INSECURE!
    if username in users_insecure_app and users_insecure_app[username] == password:
        return jsonify({"message": f"Login successful for {username}!"}), 200
    else:
        # Exposes whether username exists or not (enumeration attack vulnerability)
        return jsonify({"message": "Invalid username or password"}), 401

@app.route('/', methods=['GET'])
def home_insecure():
    return "Welcome to the Insecure Flask Login API! Use POST /login with JSON payload for username and password."

def run_flask_app():
    app.run(host='0.0.0.0', port=5000, debug=False)

# Start Flask in a separate thread
flask_thread = threading.Thread(target=run_flask_app)
flask_thread.daemon = True # Allow main program to exit even if thread is running
flask_thread.start()

print("Insecure Flask app started on http://0.0.0.0:5000")
```

```
No process found running on port 5000.
Insecure Flask app started on http://0.0.0.0:5000
 * Serving Flask app '__main__'
```

```python
import requests
import json

base_url = "http://127.0.0.1:5000"
login_url = f"{base_url}/login"

headers = {'Content-Type': 'application/json'}

print("\n--- Testing Insecure API (Successful Login) ---")
success_payload = json.dumps({"username": "testuser", "password": "password123"})

try:
    response = requests.post(login_url, headers=headers, data=success_payload)
    print(f"Status Code: {response.status_code}")
    print(f"Response: {response.json()}")
except requests.exceptions.ConnectionError as e:
    print(f"Could not connect to Flask app. Make sure it's running: {e}")

print("\n--- Testing Insecure API (Failed Login - Wrong Password) ---")
fail_payload_pass = json.dumps({"username": "testuser", "password": "wrongpass"})

try:
    response = requests.post(login_url, headers=headers, data=fail_payload_pass)
    print(f"Status Code: {response.status_code}")
    print(f"Response: {response.json()}")
except requests.exceptions.ConnectionError as e:
    print(f"Could not connect to Flask app. Make sure it's running: {e}")

print("\n--- Testing Insecure API (Failed Login - Non-existent User) ---")
fail_payload_user = json.dumps({"username": "nonexistent", "password": "anypass"})

try:
    response = requests.post(login_url, headers=headers, data=fail_payload_user)
    print(f"Status Code: {response.status_code}")
    print(f"Response: {response.json()}")
except requests.exceptions.ConnectionError as e:
```

Insecure Output:

```
INFO:werkzeug:127.0.0.1 - - [31/Mar/2026 12:02:06] "POST /login HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [31/Mar/2026 12:02:06] "POST /login HTTP/1.1" 401 -
INFO:werkzeug:127.0.0.1 - - [31/Mar/2026 12:02:06] "POST /login HTTP/1.1" 401 -

--- Testing Insecure API (Successful Login) ---
Status Code: 200
Response: {'message': 'Login successful for testuser!'}

--- Testing Insecure API (Failed Login - Wrong Password) ---
Status Code: 401
Response: {'message': 'Invalid username or password'}

--- Testing Insecure API (Failed Login - Non-existent User) ---
Status Code: 401
Response: {'message': 'Invalid username or password'}
```

Improvised Prompt:

"#Generate a Python web service for login, identify security vulnerabilities, and refactor it using secure coding practices like input validation and password hashing."

secure Code:

```
!pip install Flask bcrypt
```

```
Requirement already satisfied: Flask in /usr/local/lib/python3.12/dist-packages (3.1.3)
Requirement already satisfied: bcrypt in /usr/local/lib/python3.12/dist-packages (5.0.0)
Requirement already satisfied: blinker>=1.9.0 in /usr/local/lib/python3.12/dist-packages (from Flask) (1.9.0)
Requirement already satisfied: click>=8.1.3 in /usr/local/lib/python3.12/dist-packages (from Flask) (8.3.1)
Requirement already satisfied: itsdangerous>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from Flask) (2.2.0)
Requirement already satisfied: jinja2>=3.1.2 in /usr/local/lib/python3.12/dist-packages (from Flask) (3.1.6)
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from Flask) (3.0.3)
Requirement already satisfied: werkzeug>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from Flask) (3.1.6)
```

```python
import subprocess
import threading
from flask import Flask, request, jsonify
from bcrypt import hashpw, checkpw, gensalt # Import bcrypt for password hashing
import time # Import time for adding a delay

# Helper to kill any process using the port, ensuring Flask can start.
def kill_process_on_port(port):
    try:
        command = f"lsof -t -i:{port}"
        process = subprocess.run(command, shell=True, capture_output=True, text=True)
        pids = process.stdout.strip().split('\n')

        if pids and pids[0]:
            for pid in pids:
                if pid:
                    print(f"Killing process {pid} on port {port}...")
                    subprocess.run(f"kill -9 {pid}", shell=True)
            print(f"Processes on port {port} killed successfully.")
        else:
            print(f"No process found running on port {port}.")
    except Exception as e:
        print(f"Error killing process on port {port}: {e}")

# Kill any existing Flask app on port 5000 before starting a new one (important for testing both versions)
kill_process_on_port(5000)
```

```python
app = Flask(__name__)

# --- In-memory user store (for demonstration only - NOT secure for production) ---
# Store hashed passwords instead of plain text
users_secure_app = {
    "testuser": hashpw(b"password123", gensalt()).decode('utf-8'), # Hashed password
    "admin": hashpw(b"adminpass", gensalt()).decode('utf-8') # Hashed password
}

@app.route('/register', methods=['POST'])
def register_secure():
    data = request.get_json()
    username = data.get('username')
    password = data.get('password')

    # Input validation for username and password length/content
    if not username or not password:
        return jsonify({"message": "Username and password are required"}), 400
    if not isinstance(username, str) or not isinstance(password, str):
        return jsonify({"message": "Username and password must be strings"}), 400
    if len(username) < 3 or len(username) > 20:
        return jsonify({"message": "Username must be between 3 and 20 characters"}), 400
    if len(password) < 8:
        return jsonify({"message": "Password must be at least 8 characters long"}), 400

    if username in users_secure_app:
        return jsonify({"message": "Username already exists"}), 409 # Conflict

    # Hash the password before storing
    hashed_password = hashpw(password.encode('utf-8'), gensalt()).decode('utf-8')
    users_secure_app[username] = hashed_password
    return jsonify({"message": f"User '{username}' registered successfully!"}), 201 # Created

@app.route('/login', methods=['POST'])
def login_secure():
    data = request.get_json()
```

```python
    username = data.get('username')
    password = data.get('password')

    # Basic input validation for login credentials
    if not username or not password:
        return jsonify({"message": "Username and password are required"}), 400
    if not isinstance(username, str) or not isinstance(password, str):
        return jsonify({"message": "Username and password must be strings"}), 400

    stored_hashed_password = users_secure_app.get(username)

    if stored_hashed_password and checkpw(password.encode('utf-8'), stored_hashed_password.encode('utf-8')):
        return jsonify({"message": f"Login successful for {username}!"}), 200
    else:
        # Provide a generic error message to prevent enumeration attacks
        return jsonify({"message": "Invalid username or password"}), 401

@app.route('/', methods=['GET'])
def home_secure():
    return "Welcome to the Secure Flask Login API! Use POST /register or POST /login with JSON payload for username and password."

def run_secure_flask_app():
    app.run(host='0.0.0.0', port=5000, debug=False) # debug=False for production use

# Start Flask in a separate thread
flask_thread_secure = threading.Thread(target=run_secure_flask_app)
flask_thread_secure.daemon = True # Allow main program to exit even if thread is running
flask_thread_secure.start()

time.sleep(2) # Add a 2-second delay to allow the Flask app to start

print("Secure Flask app started on http://0.0.0.0:5000")
```

```
No process found running on port 5000.
 * Serving Flask app '__main__'
 * Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
```

```python
import requests
import json

base_url = "http://127.0.0.1:5000"
register_url = f"{base_url}/register"
login_url = f"{base_url}/login"

headers = {'Content-Type': 'application/json'}

# --- Test User Registration ---
print("\n--- Testing Secure API (User Registration) ---")
new_user_payload = json.dumps({"username": "newuser", "password": "securepass123"})

try:
    response = requests.post(register_url, headers=headers, data=new_user_payload)
    print(f"Register New User Status Code: {response.status_code}")
    print(f"Register New User Response: {response.json()}")

    # Try to register the same user again (should fail)
    response = requests.post(register_url, headers=headers, data=new_user_payload)
    print(f"Register Existing User Status Code: {response.status_code}")
    print(f"Register Existing User Response: {response.json()}")

    # Test with invalid username (too short)
    invalid_user_payload = json.dumps({"username": "ab", "password": "password123"})
    response = requests.post(register_url, headers=headers, data=invalid_user_payload)
    print(f"Register Invalid User (short) Status Code: {response.status_code}")
    print(f"Register Invalid User (short) Response: {response.json()}")

    # Test with invalid password (too short)
    invalid_password_payload = json.dumps({"username": "shortpassuser", "password": "short"})
    response = requests.post(register_url, headers=headers, data=invalid_password_payload)
    print(f"Register Invalid Password (short) Status Code: {response.status_code}")
    print(f"Register Invalid Password (short) Response: {response.json()}")

except requests.exceptions.ConnectionError as e:
```

Secure Output:

```
--- Testing Secure API (User Registration) ---
INFO:werkzeug:127.0.0.1 - - [31/Mar/2026 12:04:08] "POST /register HTTP/1.1" 201 -
INFO:werkzeug:127.0.0.1 - - [31/Mar/2026 12:04:08] "POST /register HTTP/1.1" 409 -
INFO:werkzeug:127.0.0.1 - - [31/Mar/2026 12:04:08] "POST /register HTTP/1.1" 400 -
INFO:werkzeug:127.0.0.1 - - [31/Mar/2026 12:04:08] "POST /register HTTP/1.1" 400 -
Register New User Status Code: 201
Register New User Response: {'message': "User 'newuser' registered successfully!"}
Register Existing User Status Code: 409
Register Existing User Response: {'message': 'Username already exists'}
Register Invalid User (short) Status Code: 400
Register Invalid User (short) Response: {'message': 'Username must be between 3 and 20 characters'}
Register Invalid Password (short) Status Code: 400
Register Invalid Password (short) Response: {'message': 'Password must be at least 8 characters long'}

--- Testing Secure API (Successful Login) ---
INFO:werkzeug:127.0.0.1 - - [31/Mar/2026 12:04:09] "POST /login HTTP/1.1" 200 -
Success Login Status Code: 200
Success Login Response: {'message': 'Login successful for testuser!'}

--- Testing Secure API (Failed Login) ---
INFO:werkzeug:127.0.0.1 - - [31/Mar/2026 12:04:09] "POST /login HTTP/1.1" 401 -
INFO:werkzeug:127.0.0.1 - - [31/Mar/2026 12:04:09] "POST /login HTTP/1.1" 401 -
INFO:werkzeug:127.0.0.1 - - [31/Mar/2026 12:04:09] "POST /login HTTP/1.1" 400 -
INFO:werkzeug:127.0.0.1 - - [31/Mar/2026 12:04:09] "POST /login HTTP/1.1" 400 -
Failed Login (Wrong Pass) Status Code: 401
Failed Login (Wrong Pass) Response: {'message': 'Invalid username or password'}
Failed Login (Wrong User) Status Code: 401
Failed Login (Wrong User) Response: {'message': 'Invalid username or password'}
Failed Login (Missing Username) Status Code: 400
Failed Login (Missing Username) Response: {'message': 'Username and password are required'}
Failed Login (Missing Password) Status Code: 400
Failed Login (Missing Password) Response: {'message': 'Username and password are required'}
```

Explanation:

The initial web service may contain vulnerabilities like hardcoded credentials and no validation. The improved version applies secure practices such as hashing passwords and validating inputs, making the application safer and more reliable.

Task 5 – Exception Handling Improvement

Prompt:

"#Generate a Python program that takes user input and performs arithmetic operations."

Code & Output:(No Exception Handling)

```python
def simple_calculator():
    print("--- Simple Calculator (No Error Handling) ---")
    num1_str = input("Enter the first number: ")
    num2_str = input("Enter the second number: ")
    operator = input("Enter an operator (+, -, *, /): ")

    num1 = float(num1_str)
    num2 = float(num2_str)

    if operator == '+':
        result = num1 + num2
    elif operator == '-':
        result = num1 - num2
    elif operator == '*':
        result = num1 * num2
    elif operator == '/':
        result = num1 / num2
    else:
        result = "Invalid operator"

    print(f"Result: {result}")

# --- Example Usage (Insecure) ---
simple_calculator()
```

```
--- Simple Calculator (No Error Handling) ---
Enter the first number: 23
Enter the second number: 5
Enter an operator (+, -, *, /): /
Result: 4.6
```

Improvised prompt:

"#Generate a Python arithmetic program, identify possible runtime errors, and refactor it using proper exception handling with try-except blocks."

Error handling code:

```python
def safe_calculator():
    print("--- Safe Calculator (With Error Handling) ---")
    while True:
        num1_str = input("Enter the first number: ")
        try:
            num1 = float(num1_str)
            break
        except ValueError:
            print("Invalid input for the first number. Please enter a numeric value.")

    while True:
        num2_str = input("Enter the second number: ")
        try:
            num2 = float(num2_str)
            break
        except ValueError:
            print("Invalid input for the second number. Please enter a numeric value.")

    while True:
        operator = input("Enter an operator (+, -, *, /): ")
        if operator in ['+', '-', '*', '/']:
            break
        else:
            print("Invalid operator. Please choose from +, -, *, or /.")

    try:
        if operator == '+':
            result = num1 + num2
        elif operator == '-':
            result = num1 - num2
        elif operator == '*':
            result = num1 * num2
        elif operator == '/':
            # Specific handling for ZeroDivisionError
            result = num1 / num2
        print(f"Result: {result}")
```

Output:

```python
    except ZeroDivisionError:
        print("Error: Cannot divide by zero.")
    except Exception as e:
        # Catch any other unexpected errors
        print(f"An unexpected error occurred: {e}")

# --- Example Usage (Secure) ---
safe_calculator()
```

```
--- Safe Calculator (With Error Handling) ---
Enter the first number: 55
Enter the second number: 36
Enter an operator (+, -, *, /): /
Result: 1.527777777777777
```

Explanation:
The initial program may crash due to invalid input or division by zero. The improved version uses exception handling to prevent crashes and display user-friendly error messages, improving reliability.

Conclusion:
This lab showed that AI-generated code can have serious security weaknesses if not reviewed properly. By applying secure coding practices like hashing, encryption, and error handling, the code becomes safer and more reliable. Overall, it highlights the importance of not blindly trusting AI and always validating and improving its output.